

Especificação Técnica: Arquitetura do TestDoc Agent com LangChain

Este documento detalha a arquitetura técnica e o modelo de implementação para o **TestDoc Agent** utilizando a biblioteca LangChain e o framework LangGraph. O objetivo central é estruturar um sistema multiagente previsível e de alta fidelidade para automatizar a documentação de estratégias de teste por funcionalidade (feature), explorando aprendizado contextual por meio de RAG e Few-shot Prompting sem a necessidade de fine-tuning.

1. Visão Geral da Arquitetura de 3 Agentes

Para otimizar o fluxo de dados e garantir a modularidade do sistema, as responsabilidades do projeto original foram consolidadas em exatamente três agentes principais operando em um grafo direcionado:

- **Agente 1: Analista de Riscos (Planner + RAG):** Realiza a análise inicial do escopo da funcionalidade, injeta exemplos de poucas demonstrações (Few-shot Prompting) via banco de conhecimento e mapeia riscos e criticidade.
- **Agente 2: Estrategista de Testes (Tool Use):** Consome os riscos estruturados e aciona ferramentas baseadas em código para aplicar a Matriz de Decisão de forma determinística, gerando os cenários de teste ideais.
- **Agente 3: Documentador e Crítico (Reflection):** Consolida o relatório final e executa um ciclo fechado de auto-revisão (crítica), higienizando o output para mitigar sugestões irrelevantes ou alucinações.

2. Estrutura do Estado Compartilhado (State)

A comunicação entre os agentes é realizada por meio de um estado unificado e imutável a cada passo, implementado via Pydantic para garantir tipagem e validação estrita dos dados em trânsito:

```
from pydantic import BaseModel, Field
from typing import List, Optional

class TestDocState(BaseModel):
    # Carga de Entrada (Input da Feature)
    feature_name: str
    description: str
```

```
business_rules: List[str]
dependencies: List[str]

# Atributos Modificados pelo Agente 1 (Analista de Riscos)
retrieved_examples: Optional[List[str]] = Field(default_factory=list)
identified_risks: Optional[List[str]] = Field(default_factory=list)
criticality: Optional[str] = None

# Atributos Modificados pelo Agente 2 (Estrategista de Testes)
recommended_test_types: Optional[List[str]] = Field(default_factory=list)
prioritized_scenarios: Optional[List[str]] = Field(default_factory=list)
justification: Optional[str] = None

# Atributos Modificados pelo Agente 3 (Documentador e Crítico)
final_documentation: Optional[str] = None
reflection_logs: Optional[List[str]] = Field(default_factory=list)
```

3. Divisão Funcional dos Nós do Grafo

3.1 Agente 1: Analista de Riscos (Planner + RAG)

Este agente atua na camada de entendimento de contexto. Ao receber a descrição de uma nova funcionalidade, ele interage com uma base vetorial (RAG) para coletar exemplos anotados de features similares, construindo dinamicamente o contexto de Few-shot Prompting. Com base nisso, ele lista os riscos de negócio e classifica a criticidade da feature.

Componente	Especificação Técnica no LangChain
Entrada Mapeada	Campos textuais básicos da funcionalidade fornecida pelo usuário.
Mecanismo Utilizado	VectorStoreRetriever integrado a um ChatPromptTemplate estruturado.
Saída Gerada	Lista de riscos potenciais e classificação categórica de criticidade (Baixa, Média, Alta, Crítica).

3.2 Agente 2: Estrategista de Testes (Tool Use)

Este componente traduz análises abstratas em planos de ação de engenharia de software. Ele utiliza o padrão de Tool Use para interagir com funções estruturadas em Python que encapsulam a Matriz de Decisão do projeto, associando as características da feature (ex: cálculos financeiros, persistência de dados) diretamente aos tipos de teste recomendados e gerando os cenários estruturados.

Componente	Especificação Técnica no LangChain
Entrada Mapeada	Estado consolidado com riscos e criticidade injetados pelo Agente 1.
Mecanismo Utilizado	LLM acoplado via <code>`llm.bind_tools()`</code> apontando para funções nativas de decisão.
Saída Gerada	Matriz mapeada com tipos de testes (Unitário, Integração, E2E), cenários e justificativa técnica.

3.3 Agente 3: Documentador e Crítico (Reflection)

Responsável pela formatação técnica e pelo controle de qualidade. O agente compila todos os dados coletados no layout estruturado do plano de testes. Antes de finalizar, ele dispara um gatilho interno de auto-revisão (Reflection Pattern) para verificar se o plano gerado viola alguma restrição de QA, como a inclusão de testes irrelevantes (ex: sugerir testes de performance complexos para componentes estáticos).

Componente	Especificação Técnica no LangChain
Entrada Mapeada	Todos os dados cumulativos de riscos, testes, cenários e justificativas.
Mecanismo Utilizado	Prompt de crítica reflexiva com arestas condicionais (Conditional Edges) do

Componente	Especificação Técnica no LangChain
	LangGraph.
Saída Gerada	Documento técnico final de testes limpo e validado.

4. Orquestração de Fluxo com LangGraph

A coordenação do fluxo de ponta a ponta é estabelecida mapeando os agentes como nós conectados. Caso o Agente 3 identifique falhas graves durante a auto-reflexão, o grafo direciona o fluxo de volta para o nó estrategista de forma iterativa antes de atingir o ponto final ('END'):

```
from langgraph.graph import StateGraph, END

# Inicialização do grafo baseado na classe de Estado Pydantic
workflow = StateGraph(TestDocState)

# Inclusão dos nós dos 3 agentes especializados
workflow.add_node("analista_riscos", agent_1_risk_analyst_node)
workflow.add_node("estrategista_testes", agent_2_test_strategist_node)
workflow.add_node("documentador_critico", agent_3_documenter_reflection_node)

# Definição das conexões sequenciais obrigatórias
workflow.set_entry_point("analista_riscos")
workflow.add_edge("analista_riscos", "estrategista_testes")
workflow.add_edge("estrategista_testes", "documentador_critico")

# Lógica de Controle Condicional (Laço de Reflexão)
def evaluate_reflection_path(state: TestDocState):
    if "REVER_ESTRATEGIA" in state.reflection_logs[-1]:
        return "estrategista_testes"
    return END

workflow.add_conditional_edges(
    "documentador_critico",
    evaluate_reflection_path
```

```
)
```

```
# Compilação da aplicação final  
app = workflow.compile()
```

5. Alinhamento com Critérios Acadêmicos e Métricas

Essa arquitetura multiagente responde diretamente ao problema de pesquisa científica proposto no projeto. Ao segmentar as tarefas, é possível isolar e testar o impacto de cada alteração (como desativar o RAG ou alterar os exemplos de Few-shot), permitindo calcular precisamente a variação de métricas quantitativas como F1-score, Precisão e Recall em relação à baseline fixa estipulada.